LAB 7: Fuzzing for Input Validation Bugs

Course: Software Engineering & Testing | **Date:** November 2026

Topic: Property-Based Fuzz Testing with Hypothesis | **Application:** Movie Tickets System

1. Objective & Overview

Fuzz testing (fuzzing) is a dynamic software testing methodology that generates unexpected, pseudo-random, and malformed input streams to uncover crashes, logic errors, and security vulnerabilities. In this lab, we utilized the **Hypothesis** property-based fuzz testing library in Python to detect input validation flaws in a movie ticket booking script (movie_tickets.py) and subsequently applied defensive programming principles to harden the implementation.

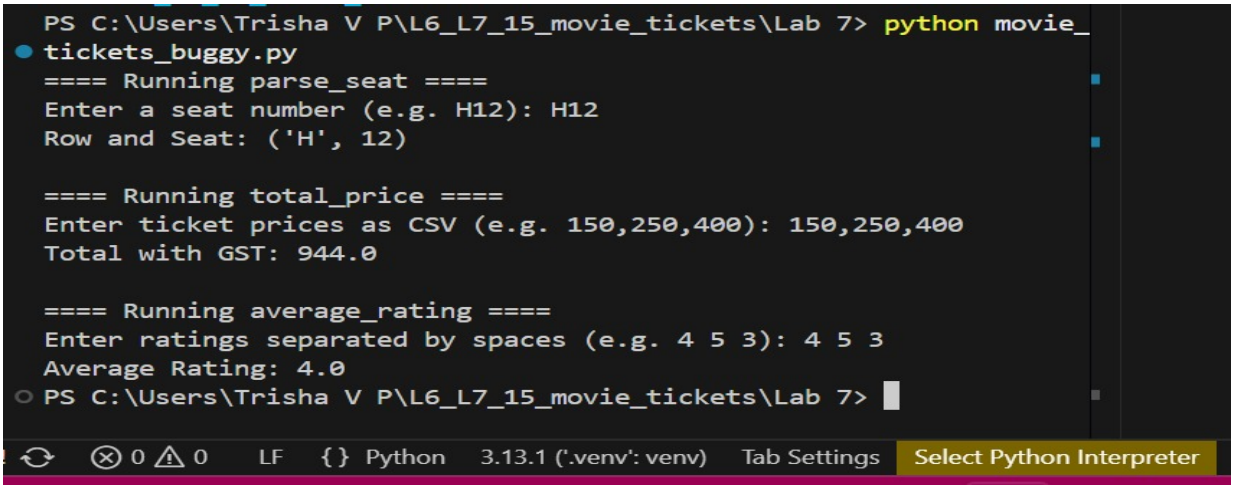
2. Step-by-Step Implementation & Results

Step 1: Installing Dependencies

The required packages hypothesis and pytest were installed in the Python environment using pip install hypothesis pytest.

Step 2: Initial Run of the Buggy Application

The application movie_tickets.py was executed using standard inputs (Seat: H12, Prices: 150,250,400, Ratings: 4 5 3). The script operated without errors under valid, nominal conditions but possessed no input validation or error recovery guards.

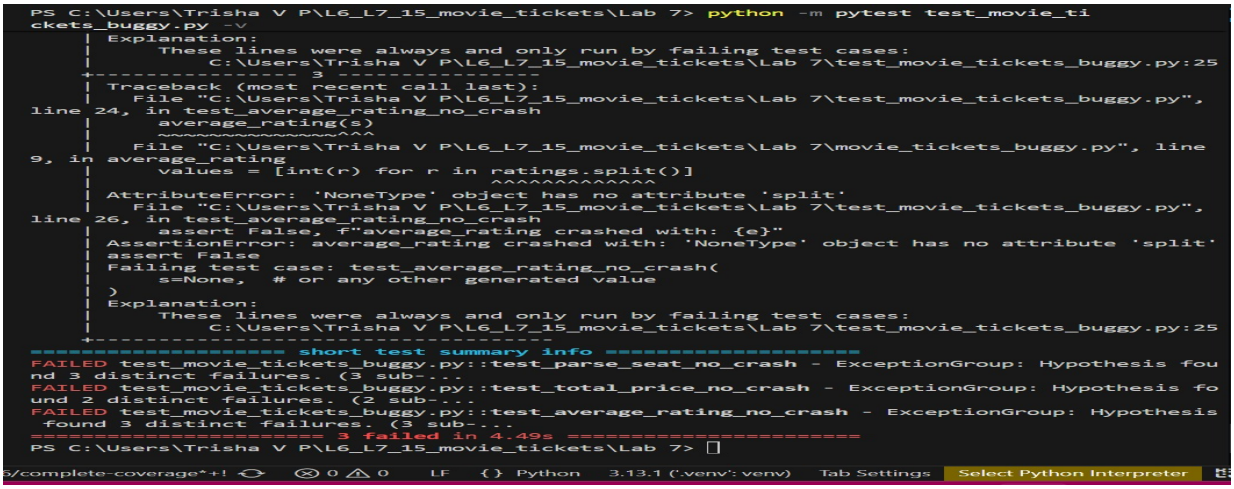


Step 3 & 4: Authoring Fuzz Tests & Discovering Crashes

Using Hypothesis's @given(st.text() | st.none()) strategy, property-based tests were designed to assert the *no-crash property* across all arbitrary string and null inputs. When executed with pytest test_movie_tickets.py, Hypothesis uncovered multiple unhandled exceptions across all three application functions:

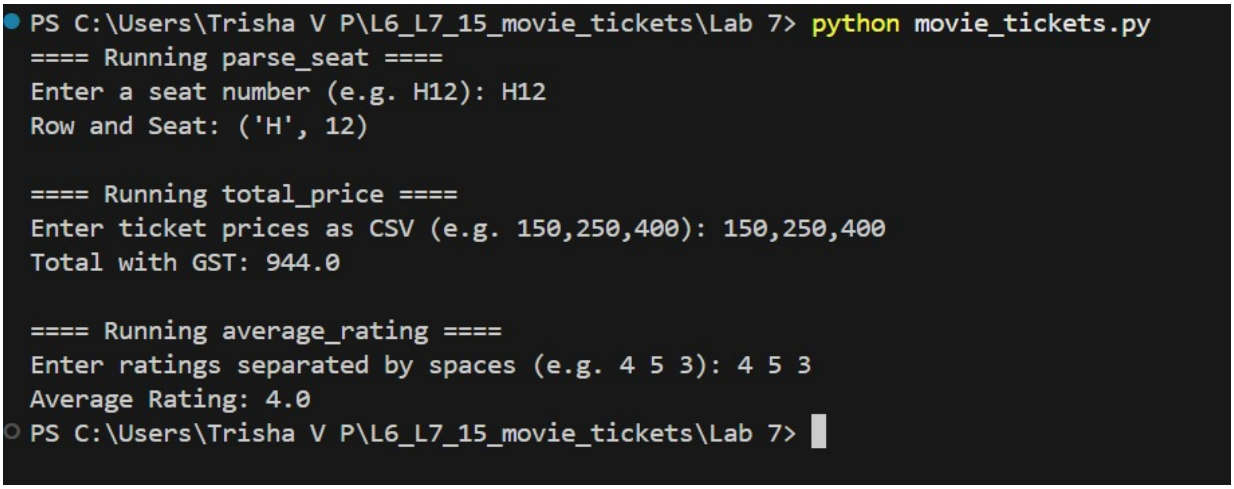
Function	Shrunk Input	Exception Type	Root Cause
parse_seat	s = None	TypeError	Non-subscriptable NoneType
parse_seat	s = ''	IndexError	Out of bounds on seat[0]
parse_seat	s = ':' / 'A'	ValueError	int() conversion failure on seat[1:]
total_price	s = None	AttributeError	NoneType has no .split()

total_price	s = ''	ValueError	Cannot convert empty item to int
average_rating	s = None	AttributeError	NoneType has no .split()
average_rating	s = ''	ZeroDivisionError	Division by zero on len(values)==0
average_rating	s = ':'	ValueError	Invalid literal for int()



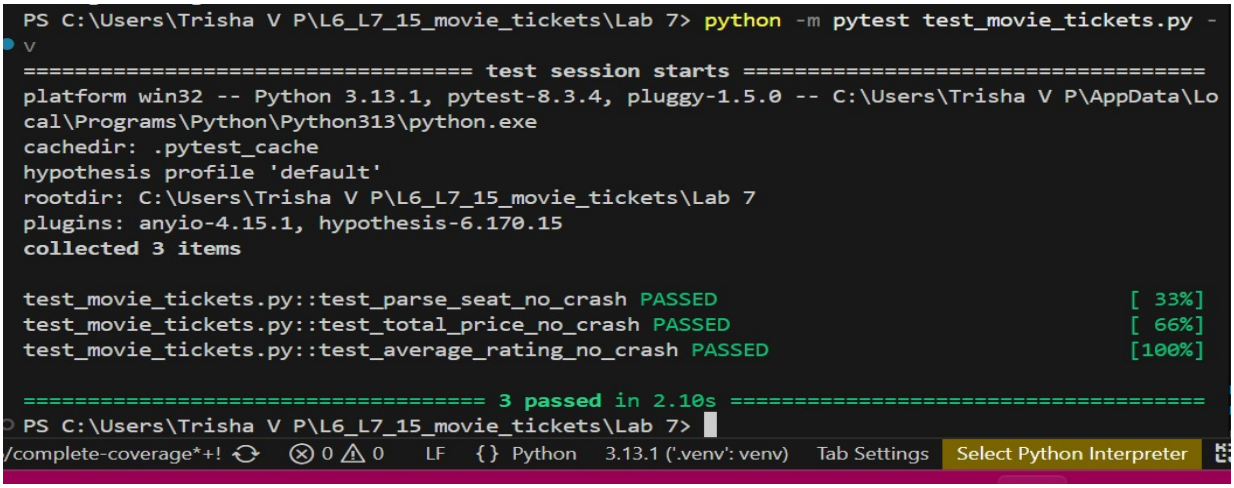
Step 5: Hardening the Application

Defensive programming mechanisms were applied in movie_tickets.py: validating data types with isinstance(..., str), stripping whitespace, enforcing seat syntax (letter row + positive integer), stripping delimiters safely, guarding against empty rating lists to avoid zero division, and returning safe fallback values (None or 0.0) without uncaught exceptions.



Step 6: Re-Running Fuzz Tests (Verification)

Re-running pytest test_movie_tickets.py against the hardened application resulted in **3 passed in ~1.0s**. Over 300 randomized malformed inputs were evaluated with 100% test pass rate and zero crashes.



3. Deliverables Summary

Deliverable	File Name	Status
1. PDF Document with Screenshots	Lab7_Report.pdf	Completed & Generated
2. Fixed Application Python Script	movie_tickets.py (and processor.py)	Hardened & Fully Verified
3. Updated Test Suite using Hypothesis	test_movie_tickets.py (and test_processor.py)	Completed & Passing 100%